

Project Design Phase – II

Requirement Analysis

Date: 10-07-2026

Team ID: SWTID-2026-5429

Project Name:

UCab – Full Stack Cab Booking System Using MERN Stack

Technology Stack (Architecture & Stack)

Technical Architecture

The UCab application follows a **three-tier architecture**, consisting of the Presentation Layer (Frontend), Application Layer (Backend), and Data Layer (Database).

- **Presentation Layer:** Built using React.js with Vite and Bootstrap, providing a responsive and user-friendly interface.
- **Application Layer:** Developed using Node.js and Express.js, which handles user authentication, booking management, fare estimation, API requests, and business logic.
- **Data Layer:** MongoDB stores user, booking, and ride information using Mongoose schemas.
- Communication between frontend and backend is achieved using RESTful APIs and Axios.
- JWT (JSON Web Token) is used for secure authentication and authorization.

Architecture Flow

User

|



React.js (Frontend)

|

Axios API Requests

|

Node.js + Express.js (Backend)

|
Business Logic

|
MongoDB Database

Table – 1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Responsive web interface for users to register, log in, and book cabs	React.js, Vite, Bootstrap, HTML5, CSS3, JavaScript
2	Application Logic – 1	User Registration, Login, Authentication	Node.js, Express.js, JWT, bcryptjs
3	Application Logic – 2	Cab Booking, Fare Calculation, Booking History	Node.js, Express.js
4	Application Logic – 3	REST API Communication and Database Operations	Express.js, Axios, Mongoose
5	Database	Stores user details, booking information, ride records	MongoDB
6	Cloud Database	Optional cloud deployment	MongoDB Atlas (Optional)
7	File Storage	Stores application files and static assets	Local File System
8	External API – 1	Future integration for live maps and navigation	Google Maps API (Future Scope)
9	External API – 2	Future payment integration	Razorpay / Stripe API (Future Scope)
10	Machine Learning Model	Not implemented in the current version	Not Applicable
11	Infrastructure (Server / Cloud)	Local development server with future cloud deployment support	Local Server, Node.js, MongoDB

Table – 2: Application Characteristics

S.No	Characteristic	Description	Technology
1	Open Source Frameworks	Frameworks used for frontend, backend, and database connectivity	React.js, Node.js, Express.js, MongoDB, Bootstrap, Axios
2	Security Implementations	Password encryption, JWT authentication, protected routes, input validation	bcryptjs, JWT, Express Middleware
3	Scalable Architecture	Three-tier architecture allows independent scaling of frontend, backend, and database	MERN Stack Architecture
4	Availability	Local deployment with support for future cloud hosting and database services	Local Server, MongoDB Atlas (Future)
5	Performance	Efficient API communication, optimized database queries, asynchronous operations	Axios, Mongoose, Node.js Async/Await

Functional Requirements

The UCab application shall provide the following functionalities:

- User Registration
- User Login
- Secure Authentication
- Cab Booking
- Pickup and Drop Location Selection
- Fare Estimation
- Display Available Cabs
- Ride Confirmation
- Booking History

- Profile Management
- Logout Functionality
- Responsive User Interface

Non-Functional Requirements

1.Performance

- The application should respond to user requests with minimal delay.
- API communication should be efficient.
- Database queries should be optimized.

2.Security

- Passwords should be securely encrypted.
- JWT tokens should protect private routes.
- Unauthorized users must not access restricted pages.

3.Reliability

- The application should maintain consistent data in MongoDB.
- CRUD operations should execute without data loss.

4.Usability

- User-friendly interface.
- Responsive design for desktop and mobile devices.
- Easy navigation.

5.Scalability

- The architecture should support adding Driver and Admin modules in the future.
- MongoDB should support increasing user and booking records.

6.Maintainability

- Modular React components.
- MVC architecture in Express.
- Reusable APIs.
- Clean folder structure.

Technology Stack

Frontend

- React.js
- Vite
- Bootstrap
- HTML5
- CSS3
- JavaScript (ES6)
- Axios
- React Router DOM

Backend

- Node.js
- Express.js
- JWT Authentication
- bcryptjs

Database

- MongoDB
- Mongoose

Development Tools

- Visual Studio Code
- ThunderClient
- Git
- GitHub

System Workflow

1. User registers or logs into the application.
2. The system verifies credentials using JWT authentication.
3. The user enters pickup and destination locations.

4. The application estimates the ride fare.
5. The user confirms the booking.
6. Booking details are stored in MongoDB.
7. Users can view booking history and ride details.
8. Users can manage their profile and log out securely.

Future Scope

- Live GPS Tracking
- Driver Dashboard
- Admin Dashboard
- Online Payment Gateway
- Google Maps Integration
- Push Notifications
- Ride Rating and Reviews
- Coupon System
- Emergency SOS Feature
- AI-based Fare Prediction

Architecture Diagram

